Fleet Simulator Engine

Overview

The goal is to implement a discrete-time simulation engine for a fleet of vessels. The engine should be driven entirely by external commands, and can track and report vessel state at each point in time.

The simulation time is driven by the tick() command; the main challenge in developing the simulation engine is determining what changes before and after a tick. I recognized that the system can be foundationally broken down into a state machine. That is, the next planned state is based on the current committed state, planned state, and the commands run by the user of the simulation engine. To better understand this, we must understand what the available commands consist of and what details a current committed state must have.

Commands

Our first command is init. Init returns the initial state at tick 0. We can think of this as the initialization of our state machine and t = 0.

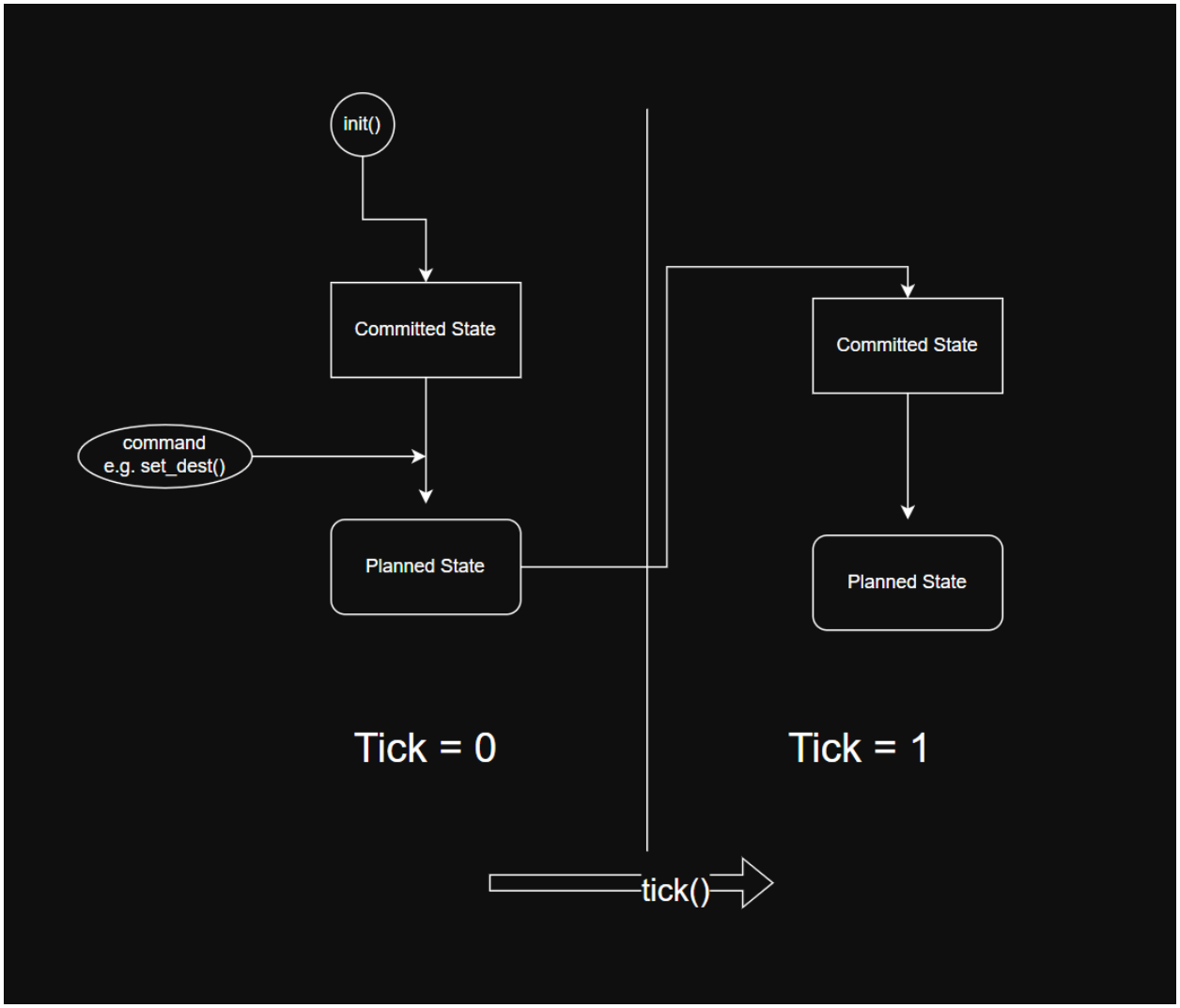
Our second command is tick. We can think of this as the positive edge of a clock input for our state machine.

The third command is set_destination. This is our main form of input to the state machine. Finally, we have get_state, our way of receiving the output from our state machine.

State

For our simulation, our committed state is represented by the current whereabouts of the vessels the system simulates. For example, a vessel can be idle, in transit, or docked. This will dictate the type of commands the vessel is allowed to receive. Planned state is necessary to record the changes that have not been committed yet.

Now that we understand our commands and state, we can understand how the simulator will function.

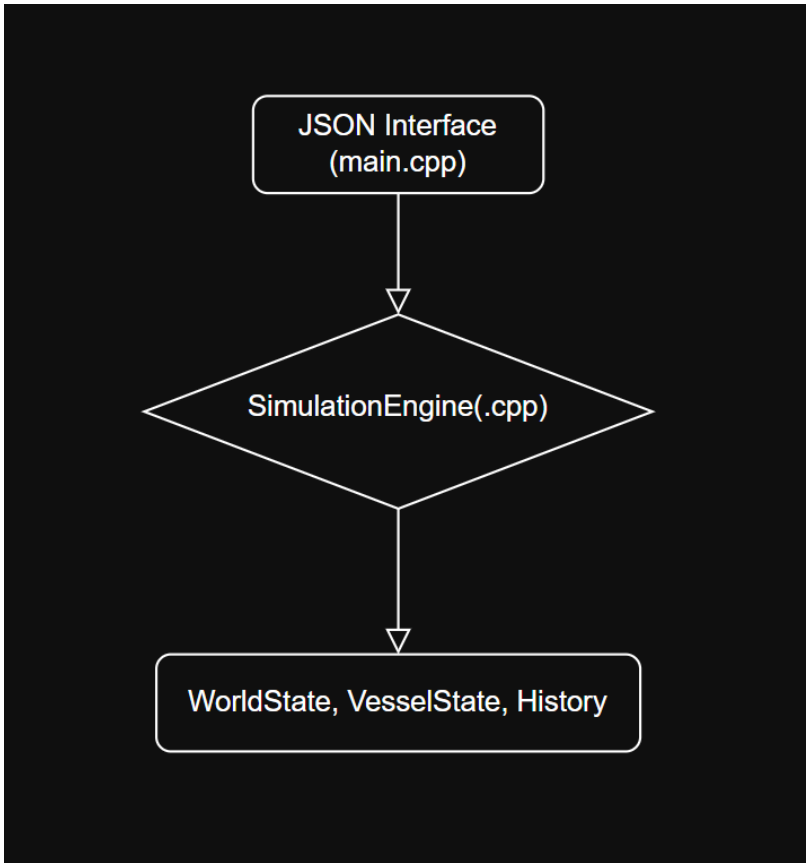


After initialization with init(), we have an initial Committed State that has our vessels at their default values. A command such as set_destination alters the planned state. Every time we tick forward, the planned state becomes the committed state, and we are now altering the next planned state with further commands. This accurately defines the basis of our system.

System Architecture

I designed my system with two main functional pieces. First, a JSON interface. This handles input parsing, type validation, and proper serialization of results. This is what the user of the simulation engine directly interacts with through the command-line terminal. The JSON interface then communicates with the second piece, which is the actual simulation engine. The simulation engine is what handles all the actual system logic. This piece implements the behavior

associated with each command, manages committed and planned states as well as the transitions between them, handles rule enforcement, and stores committed snapshots for historical queries.



Diving deeper into coding specifics, VesselState is a struct that represents an individual vessel, including vessel status, location, departure, destination, arrival tick, and cargo. WorldState is a struct that represents a snapshot of the fleet and dock inventories at a particular tick. It specifically stores the current tick, a collection of vessels, and dock cargo inventories. We use a map to hold all the vessels in a WorldState and associate vessel IDs with their states. This allows lookup by vessel ID. The SimulationEngine class owns the simulation rules, committed and planned states, travel-time configuration, dock throughput accounting, and historical snapshots. The class encapsulates the simulation's state and controls modifications. main.cpp handles the JSON protocol without directly implementing vessel behavior.

I decided to separate main.cpp from SimulationEngine because they have two distinct purposes. I can modify one without having to alter the other, and separating the components makes it easier to isolate bugs and test functionality independently.

State Management

SimulationEngine maintains two independent WorldState objects, committed and planned. The committed world represents the snapshot from the most recent tick. The planned world incorporates accepted commands issued since that tick. Both contain the simulation tick, vessel states, and dock cargo inventories. Destination commands check the committed state for eligibility but update the planned state. On a successful tick, the engine advances time, resolves arrivals, applies incoming cargo, and commits the new world.

Historical State Storage and Queries

My implementation stores complete committed snapshots after each tick. get_state_at reads these snapshots. get_state returns the current planned state. Full snapshots simplify historical retrieval, but as ticks continue to accumulate, so does memory usage.

Command Processing and Error Handling

Command Processing

This is the sequence in which command processing is implemented:

1. Read one JSON line from stdin
2. Parse JSON and validate parameter types
3. Identify the command and call its engine method
4. Check command-specific rules and constraints
5. Update planned state if the operation is valid
6. Serialize and output the response

The JSON interface handles parsing and type validation, while the engine handles simulation-specific rules. For example, main.cpp determines whether the destination value is an integer. SimulationEngine determines whether that destination exists and whether the vessel is permitted to travel.

Validation / Error Handling

There are a number of invalid commands that will be detected when attempted. These include a tick before init, an unknown vessel ID, a destination outside the valid range, a travel command while the vessel is committed to transit, loading cargo that isn't available, and exceeding dock throughput. One thing I made sure to implement was to make sure the simulation engine validates operations and uses temporary state copies where necessary to prevent partial changes.

JSON responses and failure recovery

When given a one-line input, a successful command returns the resulting world state, whereas an unsuccessful command returns an error. However, the simulator continues processing subsequent commands after a recoverable error. This is because we don't want one malformed command to terminate the entire simulation.

My implementation prioritizes correctness and predictable failure handling over minimizing allocations. Copying state before committing complicated operations costs memory and processing time, but makes it easier to guarantee that invalid commands don't partially change the simulation.

Cargo and Dock Operations

Cargo is represented using two properties. Destination describes where the cargo should be delivered. Quantity is how many units there are. VesselState stores cargo using a vector of CargoLot objects. WorldState stores dock cargo in a vector of DockCargoLot objects, which also identify the dock’s location.

The simulation supports dock, undock, load, unload, and swap commands. It is important to note that cargo operations require the vessel to be committed as docked, not just planned. Load transfers one cargo unit from dock inventory to vessel inventory. Unload transfers cargo back to dock inventory, unless the vessel is at the cargo's destination, in which case the cargo is delivered and removed. Swap performs a load and an unload together. Grouping cargo by destination avoids storing every identical cargo unit separately. It simplifies inventory management and reduces unnecessary data.

Each dock has a throughput limit that restricts how many cargo operations it can process between ticks. The simulation engine maintains an operationsUsed counter for each dock. Every successful load, unload, or swap consumes one operation. Throughput is shared between all vessels using the same dock. A command is rejected if executing it would exceed the dock's limit. Counters reset after a successful tick(). Counting accepted operations rather than net inventory changes ensures that the simulation accounts for the dock's actual workload. A load followed by an unload still consumes two operations, even if inventory returns to its original state.

During implementation, I prioritized preventing invalid operations from corrupting the simulation state, as I did in other sections. Cargo operations require a vessel to be committed as docked. The engine validates vessel status, cargo availability, destination indices, and dock throughput before accepting transfers. Complex operations such as swap must succeed completely or have no effect. The implementation prepares changes before updating the authoritative planned state. Failed commands preserve inventories, throughput counters, and vessel states. These

precautions prevent partial transfers, possible cargo duplication, and inconsistencies between dock and vessel inventories.

Running and Testing

Details for running and testing the simulator can be found in the README.